

LAB 8

Multiple Linear Regression.

Objective: Implementing Multilinear regression. In this lab, you will work with a real-world dataset "used_cars.csv", which contains information about used cars, such as car model, year, mileage, price, etc. Your objective is to:

- Clean the dataset (handle missing values, categorical variables).
- Visualize the relationship between features and the target variable (price).
- Build a Multiple Linear Regression (MLR) model.
- Evaluate the model using R^2 , adjusted R^2 , and perform an F-test.

Dataset Overview

The Used Car Price Prediction Dataset is a comprehensive collection of automotive information extracted from the popular automotive marketplace website, <https://www.cars.com/>.

This dataset comprises 4,009 data points, each representing a unique vehicle listing, and includes five distinct features providing valuable insights into the world of automobiles.

Data Dictionary

- **Brand:** Identify the brand or company name along of each vehicle.
- **Model Year:** Discover the manufacturing year of the vehicles, crucial for assessing depreciation and technology advancements.
- **Mileage:** Obtain the mileage of each vehicle, a key indicator of wear and tear and potential maintenance requirements.
- **Fuel Type:** Learn about the type of fuel the vehicles run on, whether it's gasoline, diesel, electric, or hybrid.
- **Accident History:** Discover whether a vehicle has a prior history of accidents or damage, crucial for informed decision-making.
- **Price:** Access the listed prices for each vehicle, aiding in price comparison and budgeting.

Step 1: Data Preparation and Cleaning

- **Data Loading:** Load the provided used cars dataset and display the first few rows to understand its structure.
- **Data Cleaning:** Clean the dataset by removing missing values etc.
- **One-hot encoding of Categorical variables:** Some features in the dataset might be categorical (e.g., car model, fuel type). Use one-hot encoding to convert these categorical variables into numeric variables (refer below)

Step 2: Data Visualization

- Use scatter plots or pair plots to visualize the relationships between different feature variables and the target variable price.
- Calculate and visualize the correlation matrix to understand the relationships between numerical features and check for multicollinearity.

Step 3: Building the MLR Model

- Fit the Multiple Linear Regression Model: Use LinearRegression from scikit-learn library to fit the MLR model.
- Model Coefficients and Intercept: Print the **coefficients** and **intercept** of the regression model.

Step 4: Model Evaluation

- **Model Evaluation:** Calculate the **R-squared (R^2)** and **adjusted R^2** values of the model to evaluate the goodness of fit. The formula for **Adjusted R-squared** is:

$$\text{Adjusted } R^2 = 1 - \left(\frac{(1 - R^2) \cdot (n - 1)}{n - p - 1} \right)$$

Where:

R^2 is the coefficient of determination.

n is the number of observations (data points).

p is the number of independent variables (predictors) in the model.

- **F-test:** Perform an F-test to test the overall significance of the model (use Ordinary Least Squares (OLS) from the statsmodels library) and discuss your results.

*Note: Write your interpretation for each step as required.

Explanation of Encoding Categorical Variables:

In this dataset, there are several categorical variables such as car model, fuel type and others. Since regression models require numerical input, we need to encode these variables into a format suitable for analysis. One common method is **One-Hot Encoding**.

Python Code for Encoding Categorical Variables ⁴

Python code example snippet to encode the categorical variables using One-Hot Encoding:

```
import pandas as pd

# Load the dataset
cars_data = pd.read_csv("used_cars_dataset.csv")

# List of categorical columns to encode
categorical_columns = ['brand', 'fuel_type', 'accident']

# Perform One -Hot Encoding
cars_data_encoded = pd.get_dummies ( cars_data , dtype=int , columns = categorical_columns ,
drop_first = True )

# Display the first few rows of the encoded dataset
print (cars_data_encoded.head ())

# Save the encoded dataset if needed
cars_data_encoded.to_csv ('used_cars_encoded.csv', index = False )
```

Explanation of the Code

- **Categorical Columns:** The variables listed under categorical columns need encoding because they are non-numerical and required for regression analysis.
- **One-Hot Encoding:** Each category in a variable is converted into a separate binary column. For example, if the variable `brand` has values Ford, Hyundai, Lexus, Aston, etc it will create columns like `brand_Ford`, `brand_Hyundai`, `brand_Lexus`, `brand_Aston` etc.
- **drop first=True:** This prevents the dummy variable trap by dropping one category from each variable, ensuring no redundant columns.
- **Output:** The dataset is transformed into a fully numerical format, which can then be saved for further analysis.

Result

After running the code:

- All categorical variables will be replaced with numerical dummy variables.
- The dataset is now ready for regression analysis or other statistical techniques.

Why Do We Use drop first=True?

- In One-Hot Encoding, if a categorical variable has k categories, encoding all k categories leads to perfect multicollinearity (linear dependency among variables).
- By setting `drop first=True`, we drop one category (called the reference category) and encode only $k - 1$ categories.
- This prevents the dummy variable trap, ensuring the regression model does not fail due to multicollinearity.

Does Dropping a Category Cause Data Loss?

- No, dropping one category does not lead to data loss. The dropped category is represented implicitly in the regression model's intercept and becomes the reference category, and the regression coefficients of other categories are interpreted relative to it.

When to Avoid Dropping Categories

- If you're using machine learning models that are not affected by multicollinearity, such as:
 - Tree-based models (e.g., Decision Trees, Random Forests, Gradient Boosting)
 - k-Nearest Neighbors
- In these cases, you can include all dummy variables without dropping any categories.

Summary

- Dropping one category in One-Hot Encoding does not lead to data loss; it ensures the regression model avoids multicollinearity.
- The dropped category is accounted for implicitly in the intercept and serves as the reference point for interpreting other coefficients.
- For non-linear models like Random Forests, it is safe to include all categories without using `drop first=True`